

## TEMA 5 – HOJA DE EJERCICIOS III

Se plantean diferentes ejercicios basados en modelos preentrenados de Transformers para la obtención de embeddings contextualizados.

### Ejercicio 1. Embeddings Contextualizados - BERT

A diferencia de los embeddings estáticos, que otorgan un único vector fijo a cada término independientemente del contexto en el que se encuentra, los embeddings contextualizados generan representaciones dinámicas que varían según el significado de los términos dentro de un texto concreto. Esta característica permite capturar mejor la ambigüedad y polisemia presentes en el lenguaje natural.

Uno de los modelos más representativos dentro de esta categoría es BERT. Este modelo de tipo de *encoder* utiliza una arquitectura basada en mecanismos de atención y Transformers, lo que le permite considerar el contexto completo en el que se encuentra un determinado token al generar su representación vectorial.

De este modo, BERT produce embeddings contextualizados a nivel de token en la salida de su última capa. Una vez obtenidas las salidas generadas por BERT, es posible acceder a las representaciones vectoriales de los tokens procesados mediante el atributo `last_hidden_state`:

```
outputs = model(**tokens)
outputs.last_hidden_state.numpy()
```

Dada la frase de consulta "**Dogs are domestic animals.**" y el conjunto de frases ["**Dogs are pets.**", "**This is a dog.**", "**They are free today.**"] se pide lo siguiente:

- Con el modelo y tokenizador de BERT 'bert-base-uncased', obtén el vector promedio (*mean pooling*) de la frase de consulta y el de cada frase del conjunto de frases. Nota que esta implementación de BERT utiliza los tokens [CLS], [SEP] y [PAD], por lo que se recomienda eliminar dichos tokens al hacer el vector promedio. Puedes utilizar la función del Tokenizer `tokenizer.convert_ids_to_tokens(token_id)` para localizar su posición exacta.
- Nuevamente, con el modelo y tokenizador de BERT 'bert-base-uncased', obtén el vector máximo (*max pooling*) de la frase de consulta y el de cada frase del conjunto de frases.

- Para las anteriores vectorizaciones, calcula la similitud coseno entre la frase de consulta y cada una del conjunto de frases. Reflexiona sobre qué frases son consideradas más similares por estos modelos y qué vectores funcionan mejor (media o máximo).

Nota:

- <https://huggingface.co/google-bert/bert-base-uncased>
- <https://numpy.org/doc/2.2/reference/generated/numpy.mean.html>
- <https://numpy.org/doc/2.2/reference/generated/numpy.max.html>

## Ejercicio 2. Embeddings Contextualizados – SBERT

Recientemente han surgido diversas alternativas basadas en la arquitectura original de BERT para el cálculo de embeddings contextuales. Una de estas alternativas es SBERT, adaptación que utiliza redes siamesas / bi-encoders específicamente fine-tuneados para tareas de recuperación de la información y comparación de textos cortos.

Nuevamente, dada la frase de consulta "Dogs are domestic animals." y el conjunto de frases ["Dogs are pets.", "This is a dog.", "They are free today."] se pide:

- Con el modelo de SBERT 'all-MiniLM-L6-v2', obtén los embeddings de la frase de consulta y de cada frase del conjunto de frases.
- Con el modelo de SBERT 'all-mpnet-base-v2', obtén los embeddings de la frase de consulta y de cada frase del conjunto de frases.
- Para los anteriores embeddings, calcula la similitud coseno entre la frase de consulta y cada una del conjunto de frases. Reflexiona sobre qué frases son consideradas más similares por estos modelos.
- El [Massive Text Embedding Benchmark \(MTEB\)](#) establece una comparativa sistemática entre diferentes propuestas de modelos de embeddings contextuales en función de sus características y sus propósitos. Utilizando esta comparativa, explora otros modelos basados en SBERT para la comparación de las frases analizadas.

Nota:

- <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>
- <https://huggingface.co/sentence-transformers/all-mpnet-base-v2>
- <https://huggingface.co/spaces/mteb/leaderboard>

### Ejercicio 3. Comparativa de embeddings.

Se pide, utilizando los embeddings de word2vec (word2vec-google-news-300"), generar un vector con la media de los embeddings de la frase y lo mismo para los embeddings contextuales con bert y comparar la similitud de la consulta con cada frase. ¿Qué opción va mejor?

¿Y si se hace lo mismo utilizando SBERT?

¿Habría más variación en los resultados si se utilizan frases con palabras polisémicas?

### Ejercicio 4. Medir la similitud de diferentes frases utilizando embeddings diferentes

Se proponen diferentes conjuntos de frases para probar qué embeddings funcionan mejor a la hora de medir su similitud:

#### 4.1. Frases que expresan la misma idea, pero tienen diferente orden.

sentences = [

"Don't shout at me, John.",

"Don't shout at John.",

"John, stop shouting at me."

]

#### 4.2. Frases donde hay palabras que pueden tener más de un significado

sentences = [

"The rolling Stones are rock idols.",

"Don't throw me a rock.",

"Iggy Pop is my favourite artist."

]

#### 4.3. Frases similares, pero que realmente no expresan lo mismo.

sentences = [

"I love the capital of Spain.",

"I like Madrid.",

"I love the capital of Portugal.",

"I love the capital of Japan.",

"I hate the capital of Spain.",

"I hate Japan"

]